

Desenvolvimento de serviço nativo Android com o uso de classes de serviços

CURSO

FORMAÇÃO EM SISTEMAS EMBARCADOS

LEANDRO MENDES DOS SANTOS

Links

Link do Repositório: [gitHub](#)

Link da Aplicação: [VehicleEqualizerApp](#)

Link do Vídeo: [Vídeo](#)

OBJETIVOS DE APRENDIZAGEM

- Implementar métodos do ciclo de vida dos serviços de áudio no Android;
 - Elaborar um serviço de reprodução de áudio no Android;
 - Implementar a chamada do serviço de reprodução de áudio no Android.
-

Introdução

Esté documento descreve a implementação e as funcionalidades do aplicativo de equalização. Sua integração com o AudioService e os modulos de Notificação, PlayBack e Equalização.

1. [Arquitetura](#)

- 1.1. [Estrutura de Pastas](#)
- 1.2. [Módulos](#)
 - 1.2.1. [Módulo de Reprodução \(PlaybackModule\)](#)
 - 1.2.2. [Módulo de Equalização \(EqualizationModule\)](#)
 - 1.2.3. [Módulo de Notificação \(NotificationModule\)](#)
- 1.3. [Serviço](#)
- 1.4. [Permissões](#)

2. [Definição dos Requisitos Funcionais](#)

- 2.1. [Reprodução](#)
- 2.2. [Equalização](#)
- 2.3. [Notificação](#)

3. [Referências](#)

1. Arquitetura

1.1 Estrutura de Pastas

Essa é a estrutura de pastas do projeto, vamos dar ênfase ao conteúdo de **modules** e **service**.

```
.
├── data
│   ├── AppDatabase.kt
│   ├── dao
│   │   └── ProfileDao.kt
│   ├── model
│   │   └── EqualizerProfile.kt
│   └── repository
│       └── UserRepository.kt
├── modules # Módulos com as funcionalidades de áudio
│   ├── equalizer # Responsável pela equalização do áudio
│   │   ├── EqualizerInterface.kt
│   │   └── EqualizerModule.kt
│   ├── notification # Responsável pelo gerenciamento de notificações
│   │   ├── NotificationInterface.kt
│   │   └── NotificationModule.kt
│   └── playback # Responsável pelo reprodutor de mídias
│       ├── PlaybackInterface.kt
│       └── PlaybackModule.kt
├── ProfileApplication.kt
├── service # Responsável por gerenciar serviço de áudio
│   └── AudioService.kt
├── ui
│   ├── EqualizerActivity.kt
│   ├── MainActivity.kt
│   └── viewmodel
│       ├── MainViewModelFactory.kt
│       └── MainViewModel.kt
└── util
    ├── Constants.kt
    └── ProfileRecyclerViewAdapter.kt
```

1.2 Módulos

1.2.1 Módulo de Reprodução (PlaybackModule)

O Playback Module é responsável pelo gerenciamento da execução de arquivos de áudio, abaixo, a interface (**PlaybackInterface.kts**) desenvolvida com todos os métodos utilizados para o correto funcionamento do serviço de áudio. Foi utilizado o **android.media.MediaPlayer**^[1] na classe de módulo responsável por executar os comandos básicos de execução.

```
package com.leandromendes.vehicleequalizer.modules.playback
```

```

/**
 * Playback interface
 * Interface with all methods of the playbackModule class
 *
 * @constructor Create empty Playback interface
 */
interface PlaybackInterface {
    /**
     * Set on prepared listener
     * Register a listener that will be called when the media player is
ready
     *
     * @param listener Callback
     */
    fun setOnPreparedListener(listener: () -> Unit)

    /**
     * Set raw data source
     *
     * @param resId Audio resource ID
     */
    fun setRawDataSource(resId: Int)

    /**
     * Play
     * Start playing the audio track
     */
    fun play()

    /**
     * Pause
     * Pause audio playback
     */
    fun pause()

    /**
     * Stop
     * Stop playing the audio track
     */
    fun stop()

    /**
     * Seek to
     * Changes the position of the audio track based on the position of the
SeekBar
     *
     * @param position SeekBar position
     */
    fun seekTo(position: Int)

    /**
     * Get duration
     *
     * @return Get the duration of the loaded audio track

```

```

    */
    fun getDuration(): Int

    /**
     * Get current position
     *
     * @return Get the current position of the loaded audio track
     */
    fun getCurrentPosition(): Int

    /**
     * Is playing
     *
     * @return Returns the status of the music player
     */
    fun isPlaying(): Boolean

    /**
     * Get audio session id
     *
     * @return Returns the ID of the audio section
     */
    fun getAudioSessionId(): Int

    /**
     * Release
     * Release player resource
     */
    fun release()
}

```

O método `setRawDataSource()` é responsável por criar a seção de áudio e carregar o arquivo de áudio no player. Uma callback é chamada assim que a seção está preparada, essa callback é responsável por inicializar o módulo de equalização.

```

override fun setRawDataSource(@RawRes resId: Int) {
    release()
    mediaPlayer = MediaPlayer.create(context, resId).apply {
        setOnPreparedListener {
            Log.d(logTAG, "MediaPlayer prepared")
            onPreparedListener?.invoke()
        }
        setOnCompletionListener {
            Log.d(logTAG, "Finished track.")
        }
    }
}

```

1.2.2 Módulo de Equalização (EqualizationModule)

O módulo de equalização é responsável por realizar o processamento do áudio, aplicando alteração de ganho em respectivas frequências de atuação.

A interface `EqualizerInterface()` contém todos os métodos implementados no módulo. Para realizar equalização real no arquivo de áudio, foi utilizado o `android.media.audiofx.Equalizer`[\[2\]](#).

```
package com.leandromendes.vehicleequalizer.modules.equalizer

import com.leandromendes.vehicleequalizer.data.model.EqualizerProfile

interface EqualizerInterface {

    /**
     * Apply profile
     *
     * @param profile Audio Profile
     */
    fun applyProfile(profile: EqualizerProfile)

    /**
     * Set volume
     *
     * @param level Volume value in percent [0 - 100]
     */
    fun setVolume(level: Int)

    /**
     * Set band level safe
     *
     * @param band Band ID [0 - 4]
     * @param level Gain in dB [-15 - 15]
     */
    fun setBandLevelSafe(band: Int, level: Int)

    /**
     * Set enable
     *
     * @param enabled Equalizer Status
     */
    fun setEnable(enabled: Boolean)

    /**
     * Get enabled
     *
     * @return Current status
     */
    fun getEnabled(): Boolean

    /**
     * Reset
     * Reset to default values 0db
     */
}
```

```

fun reset()

/**
 * Release
 * Enable equalizer feature
 */
fun release()

/**
 * Print bands info
 * Displays all equalizer information on the terminal, used for Log
 */
fun printBandsInfo()
}

```

Os valores de limite de ajuste das barras do equalizador estão configurados direto na SeekBar. Para descobrir esses limites, implementei os métodos. `printBandsInfo()`^[3] que exibe a quantidade de bandas do equalizador, quais frequências para cada banda e os limites máximo e mínimo de ajustes.

```

override fun printBandsInfo() {
    val bands = equalizer?.numberOfBands ?: 0
    val range = equalizer?.bandLevelRange ?: shortArrayOf(0, 0)

    Log.d(logTAG, "Equalizer has $bands bands (Min=${range[0]} dB,
    Max=${range[1]} dB)")

    for (i in 0 until bands) {
        val centerFreq = equalizer?.getCenterFreq(i.toShort()) ?: 0
        Log.d(logTAG, "Band $i → Freq: ${centerFreq / 1000} Hz")
    }
}

// Saída da função:

// Equalizer has 5 bands (Min=-15dB, Max=15dB)
// Band 0 → Freq: 60 Hz
// Band 1 → Freq: 230 Hz
// Band 2 → Freq: 910 Hz
// Band 3 → Freq: 3600 Hz
// Band 4 → Freq: 14000 Hz

```

1.2.3 Módulo de Notificação (NotificationModule)

Módulo responsável pela criação e gerenciamento da notificação persistente na barra de status. A interface `NotificationInterface()` expõe os métodos implementados no módulo de notificação.

```

package com.leandromendes.vehicleequalizer.modules.notification

```

```

import android.app.Notification

/**
 * Notification interface
 *
 * @constructor Create empty Notification interface
 */
interface NotificationInterface {

    /**
     * Build notification
     *
     * @param playbackState Current status of the player
     * @param trackTitle Audio track name
     * @return Notification class object
     */
    fun buildNotification(playbackState: String, trackTitle: String = "No
music"): Notification

    /**
     * Show notification
     *
     * @param playbackState Current status of the player
     * @param trackTitle Audio track name
     */
    fun showNotification(playbackState: String, trackTitle: String = "No
music")

    /**
     * Update notification
     *
     * @param playbackState Current status of the player
     * @param trackTitle Audio track name
     */
    fun updateNotification(playbackState: String, trackTitle: String = "No
music")

    /**
     * Cancel notification
     */
    fun cancelNotification()
}

```

1.3 Serviço

A classe `AudioService()` que estende o `android.app.Service`[\[4\]](#) do Android é responsável por inicializar e controlar todos os módulos citados acima, além disso, o `AudioService` publica atualizações para a interface do aplicativo. Vou descrever de maneira objetiva os principais métodos da classe.

O serviço é inicializado em primeiro plano[\[5\]](#) na `MainActivity` conforme abaixo:

```
// starts AudioService right at the beginning
val serviceIntent = Intent(this, AudioService::class.java)
startForegroundService(serviceIntent)
```

No método onCreate do AudioService é onde todos os módulos são iniciados.

Os módulos de notificação e playback são inicializados em sequência, já o módulo do equalizador é inicializado via Callback do playback modulo. O motivo disso é que o módulo de equalização precisa do ID da seção de áudio criada no MediaPlayer para funcionar corretamente.

```
override fun onCreate() {
    super.onCreate()
    Log.d(logTAG, "Service created")

    notificationModule = NotificationModule(this)
    playbackModule = PlaybackModule(this)

    // Only initialize the equalizer when MediaPlayer is ready
    if (playbackModule is PlaybackModule) {
        (playbackModule as PlaybackModule).setOnPreparedListener {
            initEqualizerIfNeeded()
        }
    }
}
```

Nessa seção ocorre o mapeamento dos arquivos de áudio de exemplo que serão utilizados no aplicativo.

```
trackList = listOf( // List of songs for testing
    Track(R.raw.toto_africa, "Toto Africa"),
    Track(R.raw.spain_yohan_kim_friends_concert_live, "Yohan Kim
Friends")
)
```

Após o mapeamento das músicas, elas são carregadas no player.

```
loadTrack(currentTrackIndex)
```

Por fim, duas coisas são realizadas: é iniciado o serviço de notificação com as informações e comandos do player e é iniciado o **Looper** que publica as informações para atualização das Activitys.

```
// Start notification already in stopped state
startForeground(
    NotificationModule.NOTIFICATION_ID,
    notificationModule.buildNotification(
```



```

        PlaybackStates.STOPPED,
        trackList[currentTrackIndex].title
    )
)

// Periodic updates (player position)
updateRunnable = object : Runnable {
    override fun run() {
        broadcastState()
        if (playbackModule.isPlaying()) {
            handler.removeCallbacks(this) // Avoids duplication
            handler.postDelayed(this, 1000)
        }
    }
}
}
}

```

Informações publicadas periodicamente.

```

/**
 * Broadcast state
 * states that the service sends
 */
private fun broadcastState() {
    val intent = Intent(MusicConstants.BROADCAST_MUSIC_STATE).apply {
        // State of music playback
        putExtra(MusicConstants.EXTRA_STATE, playbackState())
        // Get the current position of the song to update the counter in
        // the interface
        putExtra(MusicConstants.EXTRA_CURRENT_POSITION,
        playbackModule.getCurrentPosition())
        // obtains the total duration of the song
        putExtra(MusicConstants.EXTRA_DURATION,
        playbackModule.getDuration())
        // Current song title
        putExtra(MusicConstants.EXTRA_TRACK_TITLE,
        trackList[currentTrackIndex].title)
    }
    sendBroadcast(intent)
}
}

```

O valor de retorno escolhido foi `START_STICKY`[\[6\]](#), por garantir, caso o serviço não inicie, que ele seja reiniciado automaticamente, que no nosso caso é o ideal, pois garante consistência e todo o gerenciamento do serviço é realizado dentro da MainActivity.

```

override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int
{
    val action = intent?.action

```

```

...
// Processa as ações recebidas
...
return START_STICKY
}

```

1.4 Permissões

Permissões necessárias:

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.FOREGROUND_SERVICE"
/>
    <uses-permission
android:name="android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK" />
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS"
/>
    <uses-permission android:name="android.permission.WAKE_LOCK" />
    <uses-permission
android:name="android.permission.MODIFY_AUDIO_SETTINGS" />

```

Cadastro do serviço

```

<service
    android:name=".service.AudioService"
    android:foregroundServiceType="mediaPlayback"
    android:enabled="true"
    android:exported="false">
    <intent-filter>
        <action
android:name="androidx.media3.session.MediaSessionService"/>
        <action android:name="android.media.browse.MediaBrowserService"/>
    </intent-filter>
</service>

```

2. Definição dos Requisitos Funcionais

Estas são as duas telas do aplicativo, a primeira, responsável por exibir os perfis e gerenciar o Media Play e a lista de músicas, a segunda, responsável pela equalização, contendo todos os controles do equalizador, incluindo um botão que desativa a equalização.

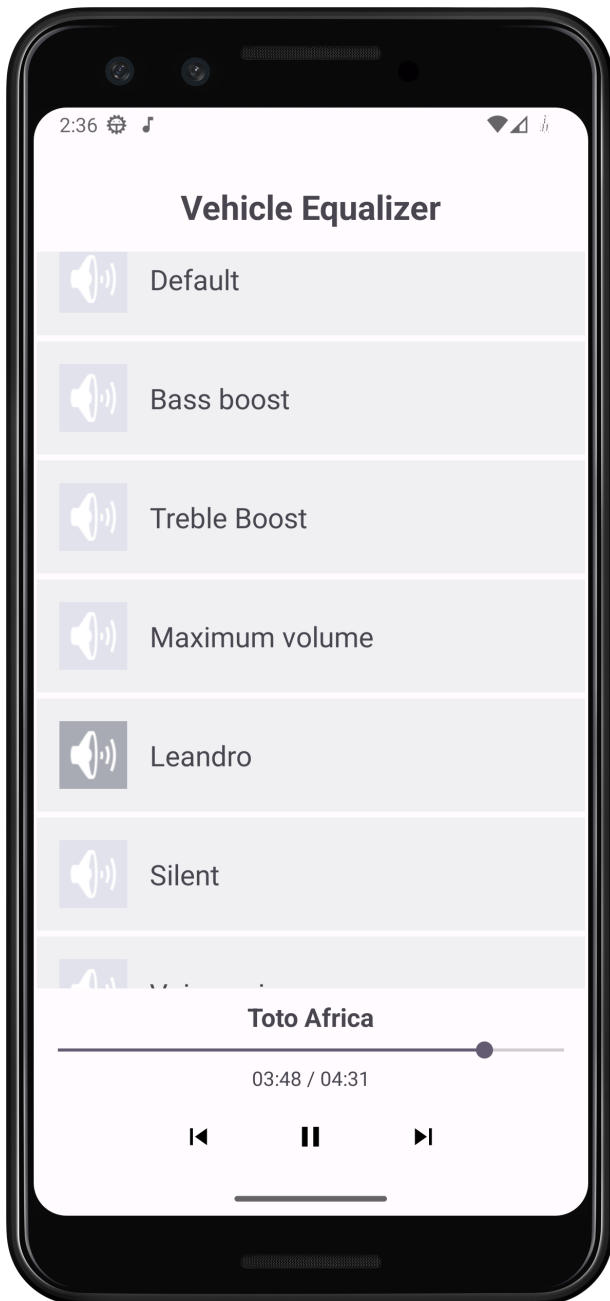


Figura 1: Tela de Perfis

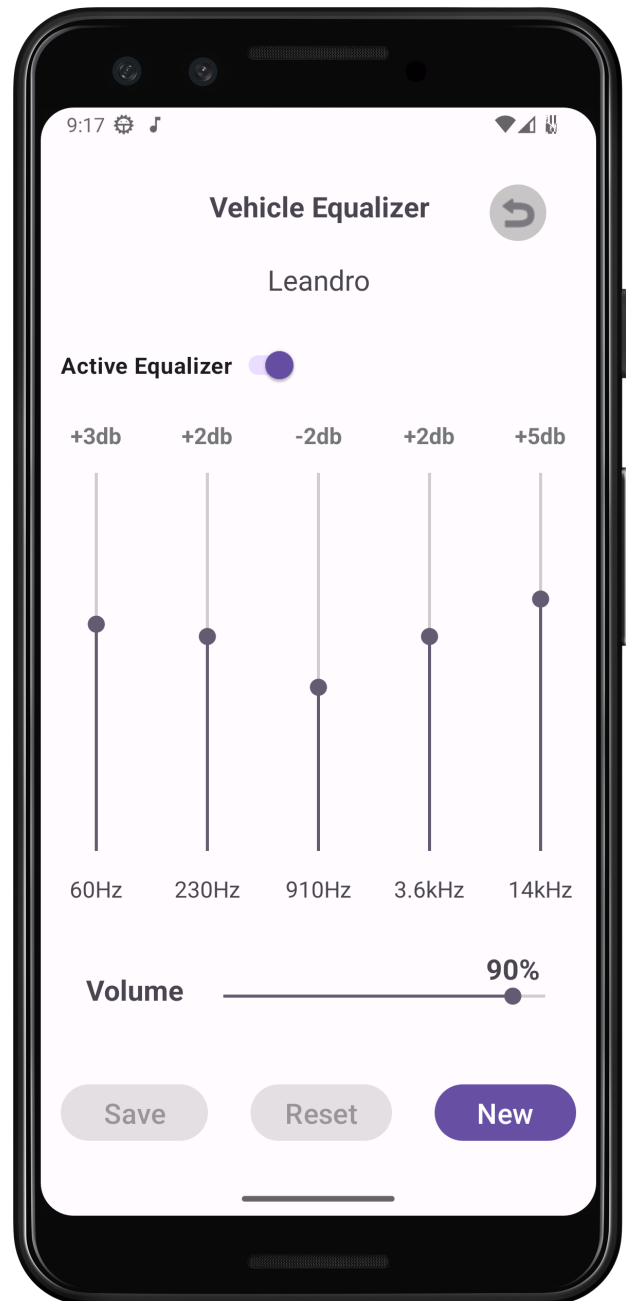


Figura 2: Tela de Edição do Equalizador

A interface foi atualizada também [Versão anterior](#), para atender as especificações do projeto. Para a implementação das SeekBars na vertical foi utilizado a biblioteca [android-verticalseekbar](#)

Um novo layout foi implementado para rodar o Aplicativo em um infotainment do Android automotive. A principal alteração do layout foi no estilo da SeekBar e no tamanho das letras. Abaixo os arquivos de layout utilizados para o Android Automotive.

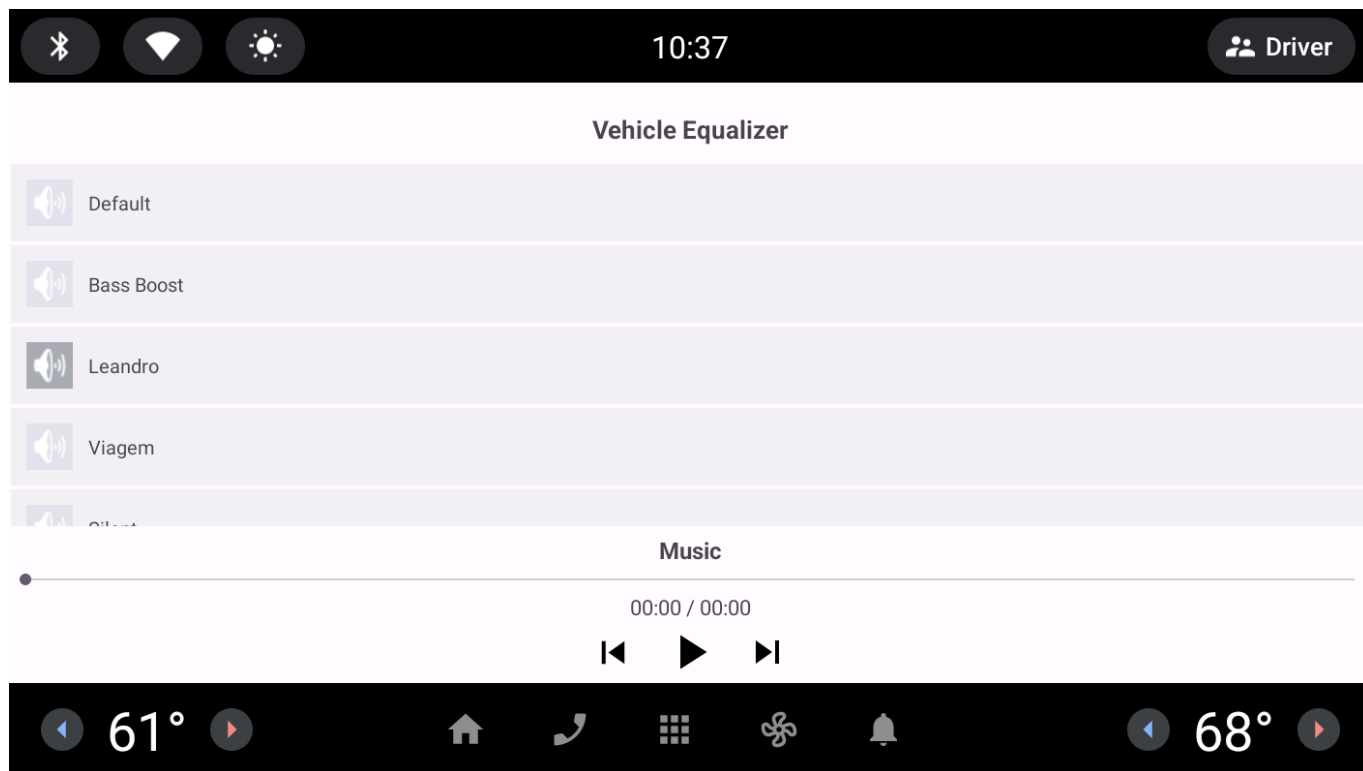


Figura 3: Tela de Perfis Android Automotive



Figura 4: Tela de Edição do Equalizador Android Automotive

Arquivos de layout modificados para a tela do infotainment.

```
res/drawable/ic_pause_.xml
res/drawable/ic_play_arrow_.xml
res/drawable/ic_skip_next_.xml
res/drawable/ic_skip_previous_.xml
```

```
res/layout/activity_equalizer_.xml  
res/layout/activity_main_.xml
```

2.1 Reprodução

Todas as principais funcionalidades do reprodutor foram implementadas, conforme a imagem abaixo, com os itens selecionados em vermelho.

Métodos implementados:

- Play
- Pause
- Próximo (Seleção de faixa)
- Anterior (Seleção de faixa)
- Busca (seek) dentro da faixa de áudio
- Exibição da faixa correspondente

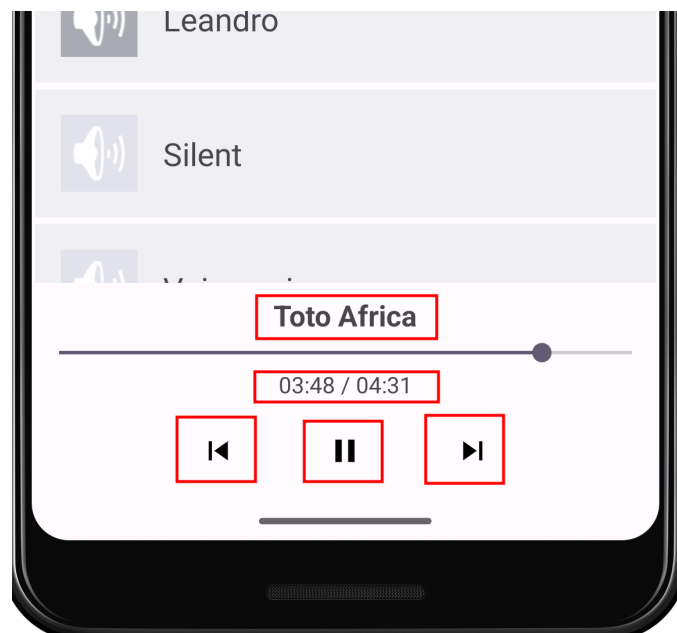


Figura 5: Seção do Player

2.2 Equalização

Na seção de equalização foram implementadas as seguintes funcionalidades:

- Equalização com 5 bandas de frequência
- Controle de volume
- On/Off do equalizador

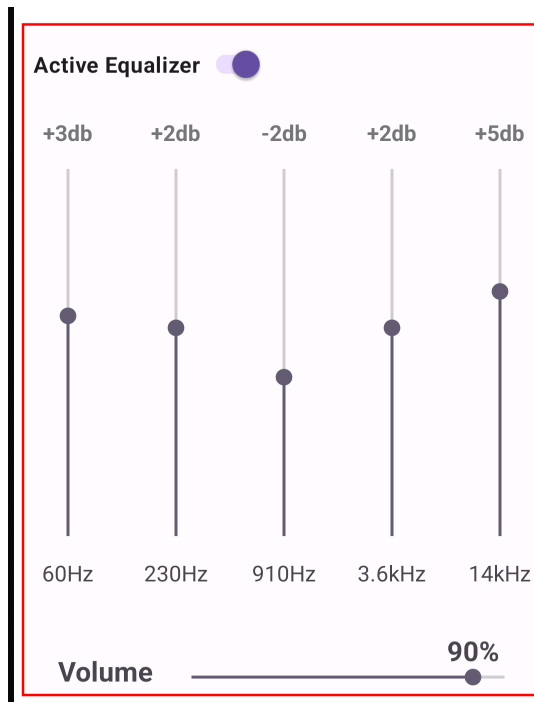


Figura 6: Seção do Equalizador

2.3 Notificação

As notificações são iniciadas logo que o aplicativo é aberto, para esta seção foram implementadas as seguintes funcionalidades:

- Play
- Pause
- Stop
- Exibe nome da música

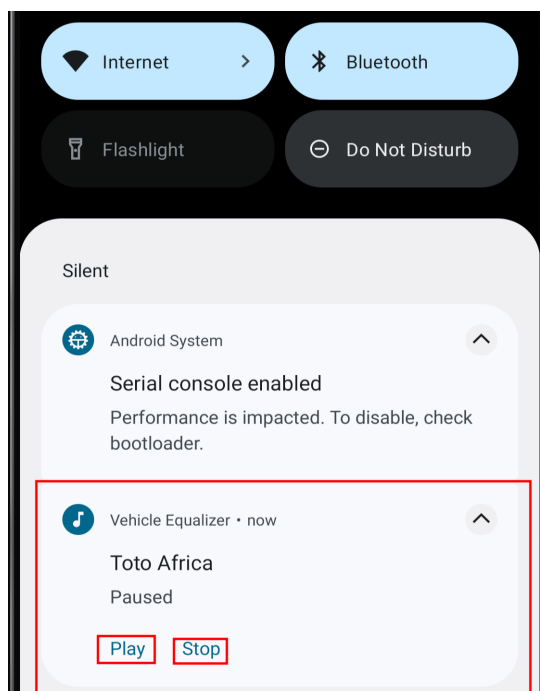


Figura 7: Seção de Notificação

3. Referências

1. [MediaPlayer](#)
2. [Equalizer](#)
3. [getCenterFreq](#)
4. [Service](#)
5. [Foreground services overview](#)
6. [START_STICKY](#)